

Multi-Agent RAG Auto Coder

Team Members: Shan Jiang (sj3475), David Sun (ds4483), Honglin Wang (hw3124), Lixiao Yang (ly2706)

Github Repository: <https://github.com/lixiao-yang/AML-Final>

Video Demo:

https://drive.google.com/file/d/1jmSaVphLju6txRzCU7c_1QJr4hdNWTmc/view?usp=sharing

Abstract

This project introduces the Multi-Agent RAG Auto Coder, a multi-agent framework designed to bridge the trust gap in AI-assisted software development by shifting from vibe-coding to a disciplined, agentic engineering workflow. By utilizing a Solver–Critic–Guider architecture grounded in Retrieval Augmented Generation (RAG) via the APPS dataset, the system moves beyond one shot code generation into an iterative Act–Observe–Reflect loop. The Solver generates hypotheses, the Critic executes code against unit tests while enforcing a heuristic "complexity gate," and the Guider synthesizes the interaction for human interpretability. Empirical results demonstrate that this decentralized approach achieves a 20% relative improvement in success rates over single-agent models, effectively reducing technical debt by catching algorithmic inefficiencies and runtime errors. Ultimately, this project highlights that role separation and closed-loop verification are essential for transforming LLMs into reliable, transparent partners in professional software engineering.

1 Introduction

Vibe-coding has rapidly emerged as a defining skill in modern software development, whether developers adopt it deliberately or implicitly. Many large technology firms now actively encourage engineers to integrate LLMs into their daily workflows to accelerate iteration speed and enable development at greater scale. However, despite these productivity gains, a critical trust gap remains. While single-agent LLMs excel at generating syntactically correct code snippets, they frequently fail on high-complexity algorithmic tasks that require multi-step reasoning, optimization, and rigorous edge-case handling. In professional settings, speed without correctness is not progress, it compounds technical debt. Empirical evidence suggests that developers may spend up to 45% more time debugging AI-generated code than code written manually, undermining the promised efficiency gains.

Bridging this gap requires a paradigm shift from *Generative AI* to *Agentic AI*: systems that do not merely generate outputs, but can verify, critique, and iteratively refine their own work. Rather than treating code generation as a one shot problem, agentic systems frame it as an ongoing engineering process with explicit feedback loops and accountability mechanisms.

Multi-agent systems have already demonstrated strong capabilities across diverse domains, including autonomous scientific discovery, financial modeling, and real time corporate strategy simulation. By replacing monolithic architectures with collections of specialized, collaborating agents, these systems achieve long horizon reasoning and complex goal fulfillment through structured interaction rather than brute-force computation.

Although real world industrial software problems are often ill-defined and continuously evolving, a reduced and well-scoped proxy can be constructed using deterministic, unit test driven programming tasks. By focusing on LeetCode-style algorithmic problems with clearly specified inputs, outputs, and correctness criteria, we transform the open-ended nature of “vibe coding” into a supervised, self correcting pipeline. Our system, referred to as the Autonomous Algorithmic Engineer, leverages this environment to move beyond one shot code generation. Instead, each initial solution is treated as a hypothesis to be tested, challenged, and refined. In this formulation, unit tests and performance constraints serve as ground truth, converting a generative modeling problem into a closed loop engineering workflow.

To further enhance robustness and domain grounding, we integrate Retrieval-Augmented Generation (RAG) into the system. RAG has gained sustained attention across multiple fields and is particularly well-suited for multi-agent pipelines, as it enables agents to reference prior knowledge, established patterns, and known solutions. By embedding retrieval directly into the agent loop, our system augments reasoning depth while reducing hallucinations and brittle implementations.

2 Related Works

One of the multi-agent applications that inspired our project is BettaFish, a public opinion analysis system designed to operate in open-ended, noisy information environments. What distinguishes BettaFish is not merely its use of multiple agents, but its deliberate separation of sense-making roles: agents independently collect data, interpret signals, and synthesize conclusions through structured internal debate. Rather than assuming correctness from any single component, BettaFish treats disagreement and reconciliation as mechanisms for reliability. This philosophy strongly influenced our system design, particularly the idea that correctness should emerge from interaction between agents with opposing incentives, generation versus critique, rather than from a single model attempting self consistency.

Complementary insights arise from general purpose multi-agent reasoning frameworks such as AutoAgents and related agentic planning systems. These works demonstrate that performance gains do not stem from repeated prompting alone, but from enforcing role isolation, memory separation, and asymmetric objectives across agents. In particular, observer or critic agents are shown to improve outcomes by evaluating artifacts from an external perspective, rather than engaging in implicit self revision. This observation directly motivated our Solver-Critic

architecture, in which execution, testing, and failure diagnosis are delegated to a dedicated Critic operating under a distinct prompt and context. Unlike prior systems that focus on open-ended planning or narrative reasoning, our project adapts these principles to deterministic, unit test driven software engineering. By combining structured agent interaction with retrieval-based grounding and executable feedback, we translate the conceptual strengths of prior multi-agent systems into a reproducible and interpretable framework for automated code synthesis and refinement.

3 Methodology

3.1 Data Acquisition and Knowledge Base Construction

To equip our agents with domain specific algorithmic intuition, we utilize the APPS (Automated Programming Progress Standard) dataset, which contains approximately 10,000 programming problems spanning introductory to championship level difficulty. APPS is a widely used benchmark for evaluating LLM-based program synthesis and closely mirrors real world coding challenges encountered in interviews and competitive programming.

For our system, we curated a high quality subset of this dataset, including verified Python reference solutions and associated unit tests, to serve as a local Knowledge Base (KB). Specifically, we selected 100 “*introductory*”, 200 “*interview*”, and 300 “*competition*” difficulty problems. These problems were stored locally and made accessible to the agents during runtime as retrieval candidates.

To support robust and reproducible retrieval, we employ a dual TF-IDF indexing strategy rather than dense embedding-based retrieval. First, we stored a word-level TF-IDF index, capturing algorithmic keywords and canonical terminology, such as “prefix sum,” “binary search,” or “dynamic programming”. Second, we stored a character-level TF-IDF index, which improves robustness to formatting differences, variable naming, and partial token overlap. Both indices are stored as sparse matrices and queried via cosine similarity. Each retrieved record is paired with metadata including difficulty level, problem constraints, and reference solution structure. This design choice prioritizes transparency, determinism, and interpretability over black box semantic embeddings.

3.2 Retrieval-Augmented Generation (RAG) Strategy

We integrate Retrieval-Augmented Generation (RAG) as a structural grounding mechanism rather than as a source of direct solution copying. When presented with a novel problem, the system performs similarity search against the local TF-IDF indices to retrieve the top- k most relevant reference problems.

These retrieved examples are injected into the Solver’s prompt as structural priors, providing guidance on: appropriate algorithmic paradigms, data structure selection, and expected computational complexity. Importantly, retrieved solutions are not treated as ground truth. Instead, they function as contextual scaffolding that constrains the Solver’s hypothesis space and reduces the likelihood of hallucinated APIs, invalid assumptions, or non-scalable approaches.

This RAG formulation improves interpretability by making the model’s reasoning traceable to explicit precedents. Each refinement step can be contextualized in terms of previously retrieved problems, allowing coding decisions to be justified with concrete, inspectable references rather than latent model activations.

3.3 Multi-Agent System Architecture

The core of our methodology is a decentralized, three-agent workflow built around the RAG-enhanced knowledge base and structured according to the *Act–Observe–Reflect* paradigm. Upon receiving an input problem, the system first retrieves the top- k most similar problems using vector-based similarity search. The input problem and retrieved references are then passed into a Solver–Critic loop.

The Solver (or Coder) is responsible for generating executable Python code. The Solver receives the problem description, a minimal code skeleton (oftentimes only containing the method signature), retrieved reference problems from the RAG module, and structured feedback from prior iterations. The Critic is responsible for executing generated code in a sandboxed environment, evaluating correctness against unit tests, and diagnosing failure modes. Execution, evaluation, and critique are unified within this agent. The Guider is responsible for post-hoc synthesis and interpretability. The Guider does not influence code generation, but instead summarizes the multi-agent interaction for human consumption.

For each problem, the Solver produces an initial implementation, which the Critic then executes against the corresponding test suite. The Critic records standard output, runtime behavior, and detailed error traces. Failures are categorized into structured failure types: wrong answer (WA), complexity error (COMPLEXITY), runtime error (RE), time limit exceeded (TLE), or system exception (exception).

Rather than performing formal static complexity analysis, the Critic relies on heuristic complexity signals, including execution timeouts, iteration patterns, and divergence from retrieved reference structures. Based on this analysis, the Critic generates a Refinement Directive, which is passed back to the Solver for the next iteration.

This loop continues until either all unit tests pass or a predefined iteration limit is reached. Upon termination, the Guider aggregates the full interaction history and produces a structured

summary, including iteration count, encountered failure types, corrective strategies attempted, and the final code’s known limitations.

3.4 Evaluation Pipeline

To evaluate the effectiveness of the Multi-Agent RAG Auto Coder, we conducted ablation studies targeting each major system component. We compared the full system against two baselines. The first is a single call, single-agent solver with no feedback or iteration, equivalent to directly prompting an LLM for a solution. The second is a repeated call, single-agent baseline that iteratively generates, tests, and refines code without introducing a separate Critic agent or distinct conversational contexts.

The key distinction between the repeated baseline and the true multi-agent system lies in agent separation. The repeated single-agent baseline performs iterative self revision within a single conversational context, whereas the multi-agent system enforces role separation through distinct prompts, memory, and objectives. This separation enables genuine agentic critique rather than implicit self editing.

To ensure reproducibility and fair comparison, we controlled all experimental parameters. The `--seed` flag was fixed to ensure consistent problem sampling across runs. The `--eval_split` parameter was set to `“test”` to prevent data leakage from the knowledge base. The `--eval_configs` flag specified difficulty levels (*introductory*, *interview*, or *competition*), ensuring balanced evaluation coverage. Additional parameters such as `--timeout` constrained execution time per problem, and `--workers` controlled parallelism for efficient evaluation.

4 Results

4.1 Evaluation Results

We used the following parameters to evaluate our multi-agent RAG model against several of its variations: `eval_n = 200`, `rag_ks = (0, 5)`, `modes = (“baseline_single”, “baseline_repeated”, “agent”)`, `repeat_n = 3`, `max_iters = 3`. Here the modes are evaluation modes or models.

Considering the agents have inherent randomness and the output may be unstable, we took the evaluation results averaged over 2 runs as our final results. The results are shown in **Table 1**, with CMP refers to “COMPLEXITY” and except to “exception” error types, abbreviated.

We can see from the table that our multi-agent model outperformed the rest, with an exciting 20%+ relative improvement over the single model and about 7.5% over the repeated baseline, with an outstanding minimal error rate amongst all types except the “exception”. With RAG, our multi-agent performance drastically improved for the baseline models and slightly for our multi-agent model, which mainly decreased its error rate.

Table 1 Evaluation of Multi-Agent RAG Model Against its Variations

Model	RAG k	success	success_rate (%)	Error Type				
				WA	TLE	CMP	RE	except
Baseline Single	0	84.0	42.0	82.0	3.0	21.5	7.0	2.5
	5	89.5	44.8	82.0	2.5	16.0	7.0	3.0
Baseline (Repeated)	0	95.0	47.5	67.5	3.5	15.0	14.5	4.5
	5	98.0	49.0	72.5	3.5	15.5	7.5	3.0
Multi-Agent (Ours)	0	103.5	51.8	67.0	1.0	14.0	11.5	3.0
	5	103.5	51.8	66.5	3.0	13.0	9.0	5.0

To demonstrate the results more intuitively, we plotted the success rate grouped by RAG k for all 3 models (evaluation modes) in **Fig. 1** as follows and the error counts (averaged) per error type via different model and RAG k pairs in **Fig. 2**. We can easily observe from **Fig. 1** that models with RAG ($k = 5$) perform better overall than those without RAG, indicating the RAG’s positive effect over the success rates. **Fig. 2** also shows a strong pattern that our multi-agent RAG model reduces the error rate most, compared with other models. The exception takes place in the error type “RE” (Runtime Error) and “exception”, which are usually caused by that agent model outputs are not always in the decodable JSON format.

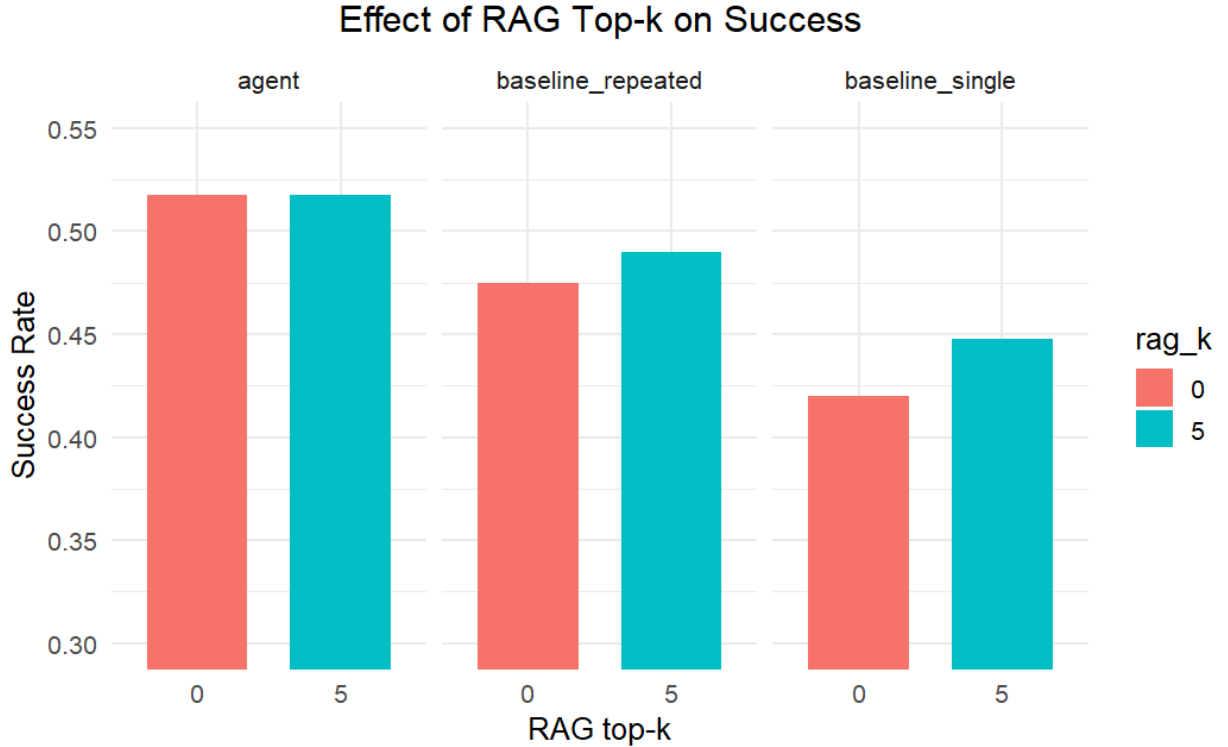
**Fig 1. The Effect of RAG Top- k on Success in 3 Evaluation Modes**



Fig 2. Error Counts per Error Type for Different Mode + RAG k Pairs

4.2 Demo

4.2.1 Frontend Details and User Interaction

The frontend of our Multi-Agent RAG Auto Coder is built using Streamlit, designed to function as a transparent "War Room" for algorithmic development. Rather than offering a simple black box generator, we prioritized observability, giving users a way to watch the system's reasoning process unfold in real time. The interface is divided into three main areas that handle control, visualization, and analysis.

The first section, Problem & Run, serves as the control center where users manage the session. Here, users can input problem JSONs directly, often retrieved using our fetch scripts, or paste in their own custom problem descriptions. This area also exposes critical system hyperparameters, allowing users to adjust the Max Iterations, set the RAG top-k retrieval depth, and select their preferred model provider, such as DeepSeek or OpenAI. This setup effectively lets the user act as a project manager, configuring the depth and intensity of the agentic debate before it begins.

At the core of the interface is the War Room / Agent Debate module, which visualizes the Act–Observe–Reflect loop. Instead of just showing the final result, the system presents a

collapsible timeline of iterations. Users can expand any specific iteration to inspect the exact state of the system at that moment. This includes the Retrieval Evidence Chain, which lists the specific problem IDs and code snippets pulled from the vector database, showing exactly what prior knowledge the Solver was referencing. Users can also view the Solver Code draft generated during that step, alongside the Critic Feedback. This feedback is presented as a structured report detailing test results, specific input/output mismatches, and the status of the "Complexity Gate," giving a complete picture of why a solution was accepted or rejected.

4.2.2 Product Justification

In professional software engineering, trusting an automated tool is difficult when you cannot see its work. Tools that simply generate code without explanation are often met with skepticism because verifying their output takes significant time and effort. Our interface addresses this trust gap by focusing on transparency rather than just the final output. Instead of hiding the process, we visualize the entire trajectory from failure to success, showing the user exactly how the system evolved from a Time Limit Exceeded error to a working solution. This proves the system is actively debugging rather than guessing and reduces the cognitive load on developers by letting them verify exactly why specific changes were made.

To further mitigate the anxiety around AI hallucinations, we ground every solution in visible evidence. The interface displays a Retrieval Evidence Chain that explicitly links generated code to known reference problems, demonstrating that the approach is rooted in established algorithmic patterns. Finally, we transform the tool from a simple code generator into a helpful partner. The Guider Agent synthesizes the entire debate into a clear, teacher-style guide, ensuring the user walks away with not just working code, but a complete understanding of the solution's logic which is essential for long-term maintenance.

5 Discussion

5.1 System Performance and Function Dynamics

Our evaluation results confirmed that breaking down coding tasks into specialized roles works significantly better than relying on a single, monolithic generation step. The Multi-Agent configuration showed a clear improvement over the single-agent baseline, proving that an iterative process of drafting and correcting is essential for solving complex algorithmic problems. However, the role of RAG was more nuanced than we anticipated. Contrary to our hypothesis, adding RAG didn't consistently improve success rates and, in some cases, even lowered them slightly. We attribute this to a "signal-to-noise" issue: while retrieving similar past problems can provide helpful structural starting points, imperfect matches often distract the Solver with irrelevant patterns. As a result, RAG currently functions more as a transparency feature, grounding the AI's thought process in known data for the user to see, rather than a raw performance booster, pointing toward a need for better filtering in future versions.

A critical success, however, was the Critic's heuristic complexity gate. This mechanism effectively simulates a senior engineer's code review by actively rejecting solutions that pass unit tests but fail to meet efficiency standards. By catching these functionally correct but slow implementations early, the system prevents the accumulation of technical debt, ensuring that code doesn't just work in a sandbox but remains viable under production loads.

5.2 Limitations and Future Improvements

Despite the system's success, we encountered several limitations that clearly define our roadmap for future work. A primary challenge is the system's heavy reliance on the LLM's ability to output strict JSON for inter-agent communication. We observed that smaller or less capable models frequently struggle to adhere to this rigid schema, leading to pipeline crashes when the output cannot be parsed. To address this, future iterations will need to implement robust "retry with repair" parsers or adopt constrained generation frameworks like Guidance or Outlines, which enforce schema compliance at the token level rather than relying on the model's adherence to instructions.

Another critical factor is the sensitivity of the Critic agent to the underlying model's reasoning capability. The system's performance hinges on the Critic's ability to accurately diagnose failures; a weak Critic that hallucinates bugs or misses subtle edge cases can actively mislead the Solver, wasting iterations on phantom errors. Optimizing the specific pairing of models could yield significantly better cost performance ratios than our current uniform approach, such as deploying a specialized "reasoner" model for the Critic alongside a faster "coding" model for the Solver.

Finally, we must address the trade-off between latency and accuracy. The iterative multi-agent loop is inherently slower than single-shot generation, which, while acceptable for complex algorithmic challenges, can feel sluggish for simpler tasks. To improve practical usability, future versions should implement a fast mechanism that bypasses the full Critic loop for trivial problems, allowing the system to balance deep reasoning with responsive performance based on the estimated difficulty of the task.

Conclusion

This project demonstrates that improving AI-assisted programming requires shifting from one shot code generation to agentic systems. The Multi-Agent RAG Auto Coder achieves this by separating code generation, evaluation, and explanation across a Solver, Critic, and Guider, enabling the system to iteratively test, diagnose, and refine its own outputs. Empirical results show that this explicit role separation outperforms both single shot and repeated single-agent baselines, with the Critic's execution-based feedback and heuristic complexity checks playing a central role in preventing incorrect or inefficient solutions. While Retrieval-Augmented Generation did not consistently improve success rates, it enhanced transparency by grounding

decisions in retrievable prior examples. Overall, this work provides a reproducible and interpretable blueprint for building AI coding systems that can detect errors, recover from failure, and earn developer trust, which are crucial for meaningful integration of LLMs into real software engineering workflows.

All members contributed equally to this project. Shan Jiang laid down the foundations and skeleton of the multi-agent system; David Sun went through related research, incorporated agent API calls, and produced evaluation reports; Honglin Wang fixed UI errors, evaluated the results, and fixed RAG implementation issues; Lixiao Yang wrote the evaluation components, including the baseline models, and the generating the evaluation results.